Visitor Pattern

An in-memory hierarchical file system

Purpose and object model

The project applies independent size, search and statistics operations to the same tree. **Element** defines name() and accept(Visitor). **FileNode** stores a non-negative byte size; **DirectoryNode** stores named children. Their shared base, FileSystemNode, controls names and parent ownership. No operation reads the physical disk.

Traversal and double dispatch

FileSystemWalker owns an iterative depth-first preorder traversal. Each node receives one typed operation call. First, element.accept selects the concrete element method. Then visitor.visit(this) selects the matching overload. A private Dispatch visitor forwards that call to the operation and schedules directory children. The three operation visitors never recurse or select behavior with type checks.

Before each walk, begin() resets results. Before each visit, enter(VisitContext) supplies the relative path and depth. Instances are reusable sequentially; concurrent operations require separate visitors. Tree mutation during traversal is unsupported. A failed walk may leave partial results; the next walk starts clean.

Paths, counts and integrity

Paths omit the chosen directory root, use /, and are sorted lexicographically with Java natural string order. A file root uses its own filename. Root depth is 0; each child edge adds 1. Empty directories contribute to directory count and depth. Search combines a configurable basename predicate with an inclusive size threshold.

Names exclude blank values, dot segments, separators and NUL. add() rejects duplicate sibling names, existing ownership, self-links and ancestor cycles before mutation. remove() releases ownership for reparenting. Child collections and results are immutable snapshots. Invalid values raise IllegalArgumentException; required null inputs raise NullPointerException. Total size uses checked addition to reject long overflow.

Executable demonstration

The workspace contains readme.txt (320 bytes), documents/notes.txt (120), documents/plan.pdf (2048), documents/archive/draft.txt (80), documents/archive/summary.txt (180), media/logo.png (4096), and an empty directory. The default search selects .txt files of at least 100 bytes.

```
Root: workspace
Total size: 6844 bytes
Files ending with .txt (minimum 100 bytes):
  documents/archive/summary.txt
  documents/notes.txt
  readme.txt
Files: 6
Directories: 5
Maximum depth: 3
```

Design and verification

Clean Code decisions, automated evidence and extension trade-offs

Five concrete Clean Code practices

Practice and reference	Justification
Meaningful naming SearchVisitor.minimumSize	Names such as minimumSize and relativePath expose the meaning of values directly.
Single Responsibility FileSystemWalker / operation visitors	Traversal and the three independent operations have separate owners and reasons to change.
Small methods FileNode.accept / StatisticsVisitor.enter	Each method performs one clear action, making dispatch and depth updates easy to inspect.
Encapsulation DirectoryNode.add/remove/children	Mutations validate invariants, while immutable snapshots prevent external collection edits.
Eliminating type checks Both accept methods / Dispatch	Overloaded visit methods select behavior without instanceof, reflection or behavior-selecting casts.

Automated verification

Local run: **mvn -B clean verify**, 8 November 2026. **45 tests passed; 0 failures, 0 errors, 0 skipped.** Maven 3.10.0 ran on JDK 25.0.2 with compiler release 17 (class-file version 61). Compilation treats warnings as errors. The executable JAR was also run with default and changed search inputs.

DemoTest: 4 tests
FileSystemWalkerTest: 4 tests
SearchVisitorTest: 7 tests
StatisticsVisitorTest: 6 tests
TotalSizeVisitorTest: 4 tests
TreeIntegrityTest: 16 tests
VisitorDispatchTest: 4 tests

Coverage includes both dispatch overloads, multilevel totals, both search constraints and the inclusive threshold, sorted root-relative paths, empty/no-match cases, repeat execution, subtree statistics, atomic invalid mutations, reparenting, overflow recovery and a 1,500-level tree. GitHub Actions is configured for JDK 17 and 25; this local result does not assert that remote CI has passed.

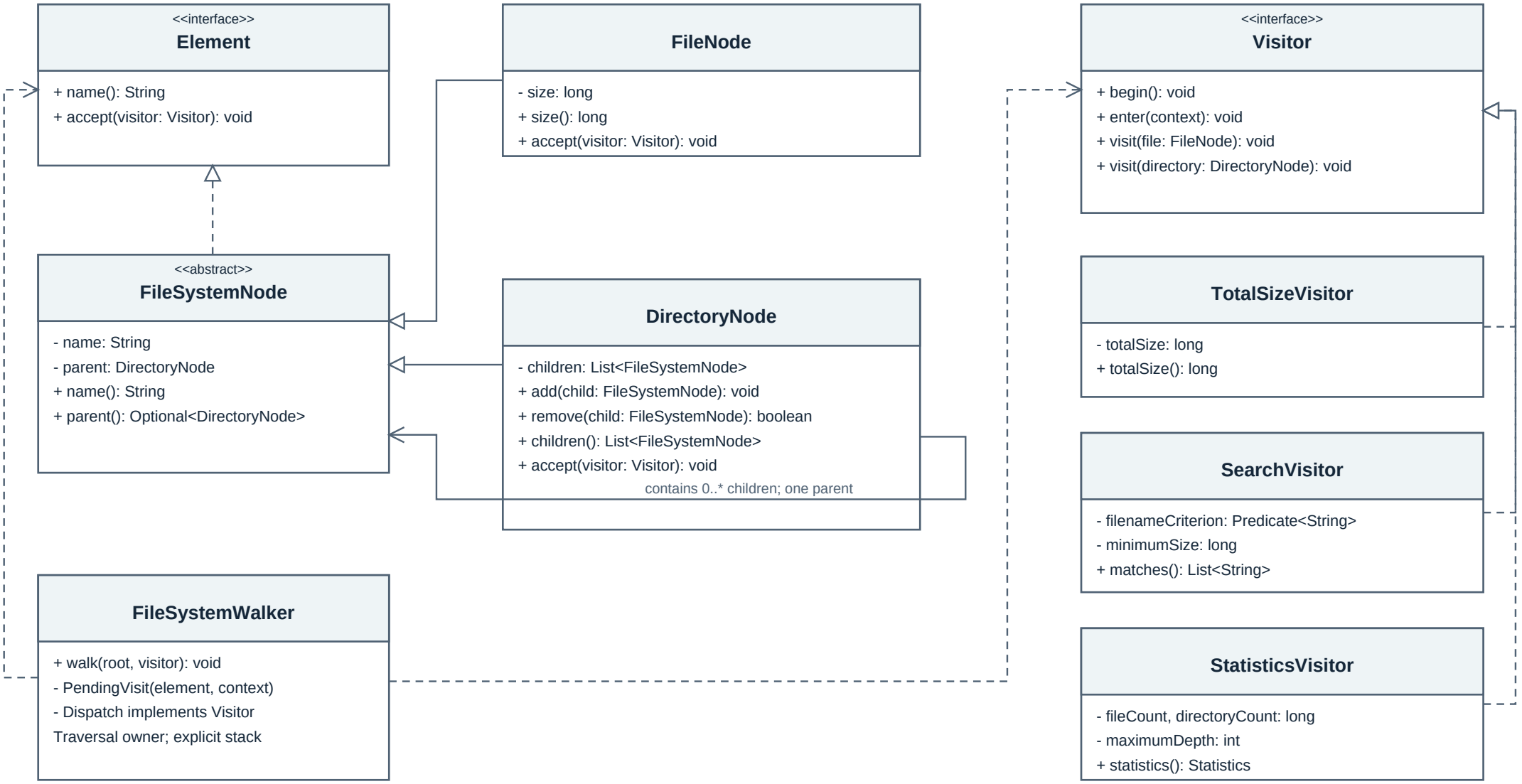
Extension boundary and trade-offs

StatisticsVisitor, Statistics and StatisticsVisitorTest were added in a separate commit after size/search and their tests. FileNode, DirectoryNode, their accept methods and the existing Visitor contract were unchanged. Adding another operation requires another visitor. Adding an element type requires a new visit overload, implementations in all visitors and a scheduling rule in the walker adapter. This design favors new operations over new types.

Run and repository

```
mvn test
mvn package
java -jar target/visitor-file-system-1.0.0.jar
```

github.com/Asylstyle/assignment6-visitor-pattern
Design basis: Assignment 06, Visitor Pattern - Advanced File System, sections 3-7.

UML class diagram

Appendix - excluded from the report page limit